Consistent Hashing

Monotonicity, Balance, Spread, and Load

Andrea Clementi

May 11, 2026

SUMMARY of the Lectures: PART I

I) (Distributed) WEB Caching

III) Traditional (Randomized) Hashing (Linear Functions)

V) Critical Issues of Item Assignment via Traditional Hashing

VI A New, Dynamic Solution: **Consistent Hashing**

PART I: A (Short and Rough) Introduction to Web Caching

What is Web Caching?

- **Definition:** Web caching is the activity of assigning copies of web resources (HTML pages, images, videos) to special Servers (i.e. **Cache Servers**).
- **The Goal:** To serve future requests for those resources faster and reduce external traffic.
- **Analogy:** It is like keeping a copy of a frequently used book on your desk instead of walking to the library every time you need to read a page.

The Problem: Why do we need it?

Web caching addresses the **Data Delivery Bottleneck**:

Specific Issues

- **Network Latency:** Data takes time to travel long distances between the client and the Origin Server.
- **Bandwidth Congestion:** Repeatedly downloading the same "viral" content clogs the network pipes.
- **Server Overload:** High demand can crash an origin server (the "Slashdot effect").

Types of Web servers

Caching happens at multiple levels:

1. **Browser server:** Stored on your local device (PC/Phone).
2. **Proxy server:** Stored at an organization/ISP level to serve many users.
3. **CDN (Content Delivery Network): Geographically distributed Cache Servers (like Akamai or Cloudflare) that store content close to users.**

Objects and Their Digital Addresses on the Internet

- A **URL** stands for *Uniform Resource Locator*. Think of it as the "digital address" of a specific resource on the internet. Just as your home address tells the post office exactly where you live, a URL tells a **Web Browser** exactly where to find a file, a page, or an image.
- Technically, a URL is a specific type of **URI** (*Uniform Resource Identifier*) that not only identifies a resource but also describes the primary access mechanism (the **Protocol**) used to *retrieve* it.

The Basic Workflow

1. **Request:** User requests a URL (e.g., `image.jpg`).
2. **Intercept:** The **Cache** checks if it has a local copy.
3. **Cache Hit:** If found and fresh, the Cache sends the copy immediately. *(Very Fast!)*
4. **Cache Miss:** If not found, the Cache fetches it from the original server, stores a copy, and then gives it to the user.

Elementary Example: The "Logo" Case

Imagine a website with 1,000 pages. Every page displays the same **logo.png** (1MB).

- **Without Caching:** If a user visits 10 pages, they download 10MB of data for the same image.
- **With Caching:** The user downloads 1MB once. For the next 9 pages, the browser loads it from the disk in milliseconds.
- **Result:** 90% reduction in traffic and instant page loads.

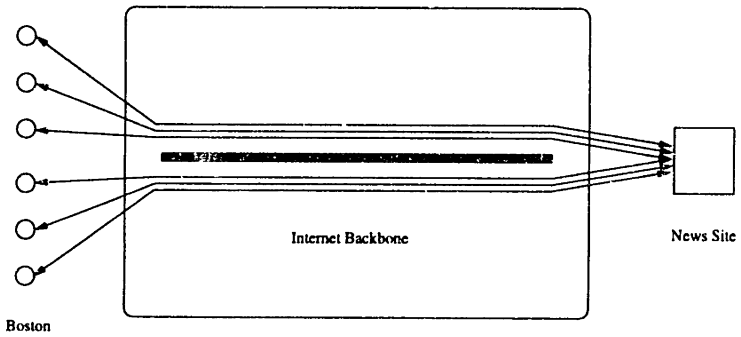
The Distributed Caching Model

Decentralized Caching System

- The **Cache System** is formed by a large set of small **Cache Servers (CS)** that are spread over the Network (Internet) so that every *User* has some CS "close" to him.
- **Strategic Goal:** CS cooperate to form a powerful, General Cache System that can efficiently manage user requests.

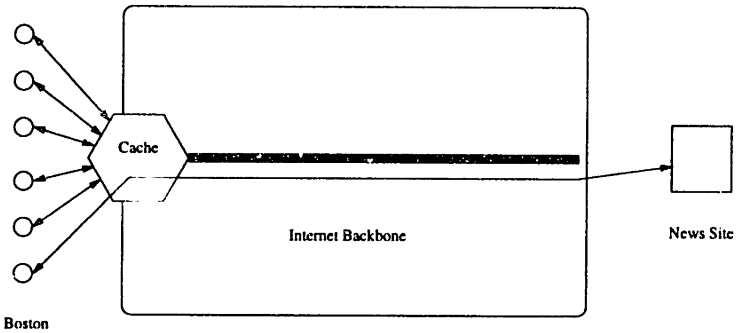
Web Caching System

CENTRALIZED STRATEGY



(i)

DISTRIBUTED CACHING STRATEGY



(ii)

Figure 1.1: (i) Multiple users in Boston ...

Distributed Web Caching System

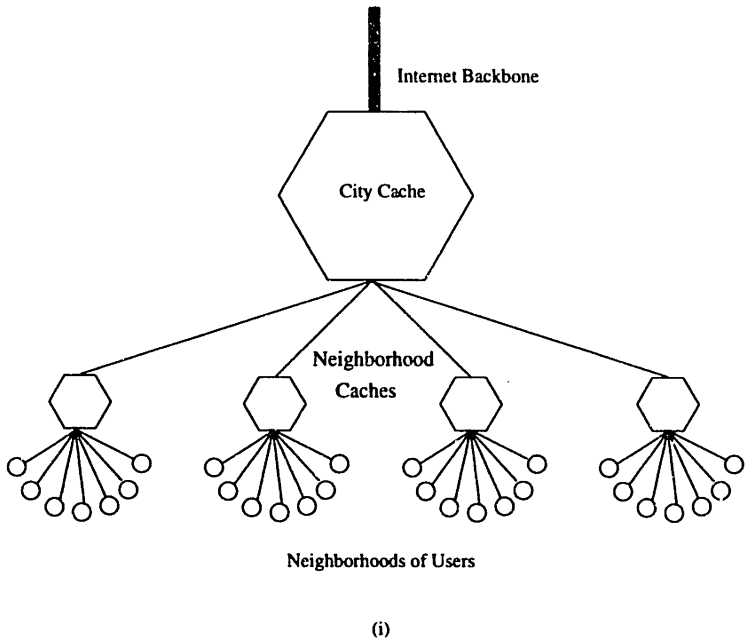


Figure 1-3: (i) A hierarchy of caches with small neighborhood caches at the bottom of

The Distributed Caching Model

Technical Objectives

- **Managing a Large Distributed System**
- **Decentralized Dynamic Control**
- **Robust Under Imperfect and Inconsistent Information**
- **Scale Gracefully**
- **Prevent CS Swamping and Hot Spots**
- **Minimize Network Usage**
- **Balance Storage Requirements**
- **Low Overhead Time for User Queries**

Decentralized Caching System

Our Specific Focus

- Design an High-level Data Structure that manage *How* Items (e.g. Web Pages) are distributed and replicated to Caching Servers (CS) so that some specific **Properties** are theoretically guaranteed.
- Such Properties globally play a key-role to achieve the *Technical Objectives* of *Decentralized Caching Systems* discussed in the previous slide
- This Data Structure is called **Consistent Hashing**

How do we find data in the Cache?

- A cache stores thousands (or millions) of items.
- We need a way to **uniquely and quickly identify** which Cache Server is responsible for each specific item.
- **The Solution: Hash Functions**

Our Simplfied Cache-System Model (CSM)

CSM: Ingredients

- A set I of **Items**
- A set B of **(Cache) Servers:** Contains **Items** (i.e. WEB Pages)
- A set U of **Users:** Agents that want to retrieve Items from the Servers

CSM: Specific Goal

- **Hypothesis:** All Items are accessed by Users at apx the same rate, *however*, Some Servers may have far more (Item-)**Load** than others and, so, much more User Queries to answer.
- **Goal**: To **Spread** the Load of User Queries *almost uniformly* over the Server Set (**Servers** are acting as **Caches**)

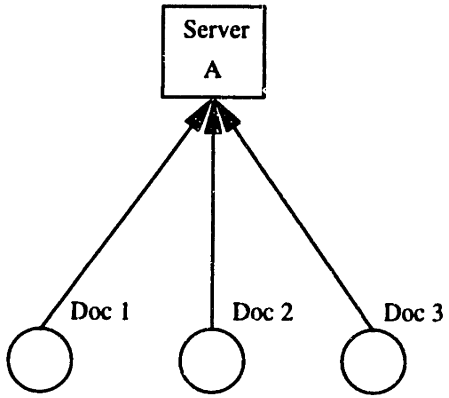
Our Simplfied Cache-System Model (CSM)

CSM: Distributed Solution via **Hash Functions**

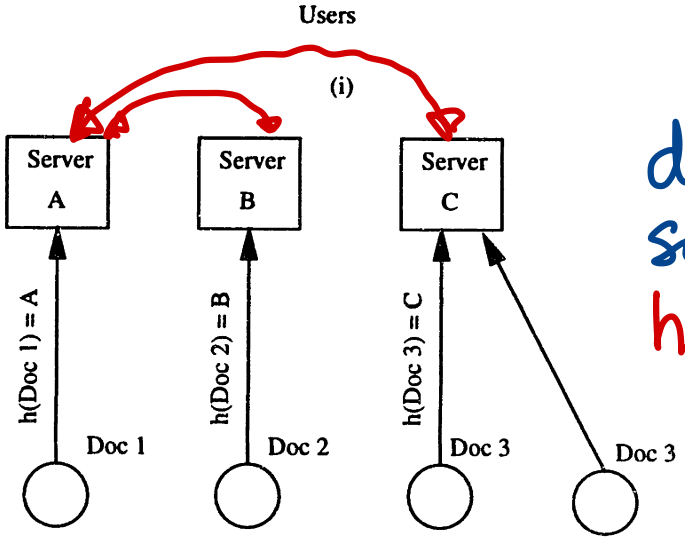
- Every Item is assigned to a Server via a **Hash Function** $f : I \to B$
- A Server b is said to be **Responsible** for item i if $\mathbf{f(i) = b}$
- Each User u knows f and asks x to Server $b = f(i)$ (instead of the original server b^o): If b has a copy of i it gives back to u. If not, b asks a copy to b^o (via Internet) and send it to u.
- Notice that *any* subsequent user request for x will be satisfied by $b = f(i)$ *without involving* b^o (no use of Internet).
- The Hash Function $f : I \to B$ is shared by *all* users and, indeed, it serves as a method *to agree* on which server is responsible for any item without communicating.

Distributed Web Caching System via Hashing

NOTE : hash function h must be shared by all users and servers



(i) centralized solution



(ii) distributed solution via h2sh functions h

Utilizing Hash Functions

A Hash Function (like MD5 or SHA-1) takes a URL and turns it into a fixed-length *Fingerprint*.

$$\text{URL} \rightarrow \text{Hash Function} \rightarrow \text{Hash Key (e.g., 64-bit integer)}$$

Why use hashes?

- **Fixed Size:** All keys have the same length regardless of URL length.
- **Fast Lookup:** Used as an index in a *Hash Table* ($O(1)$ complexity).
- **Efficiency:** Comparing two integers is much faster than comparing two long strings.
- BUT....

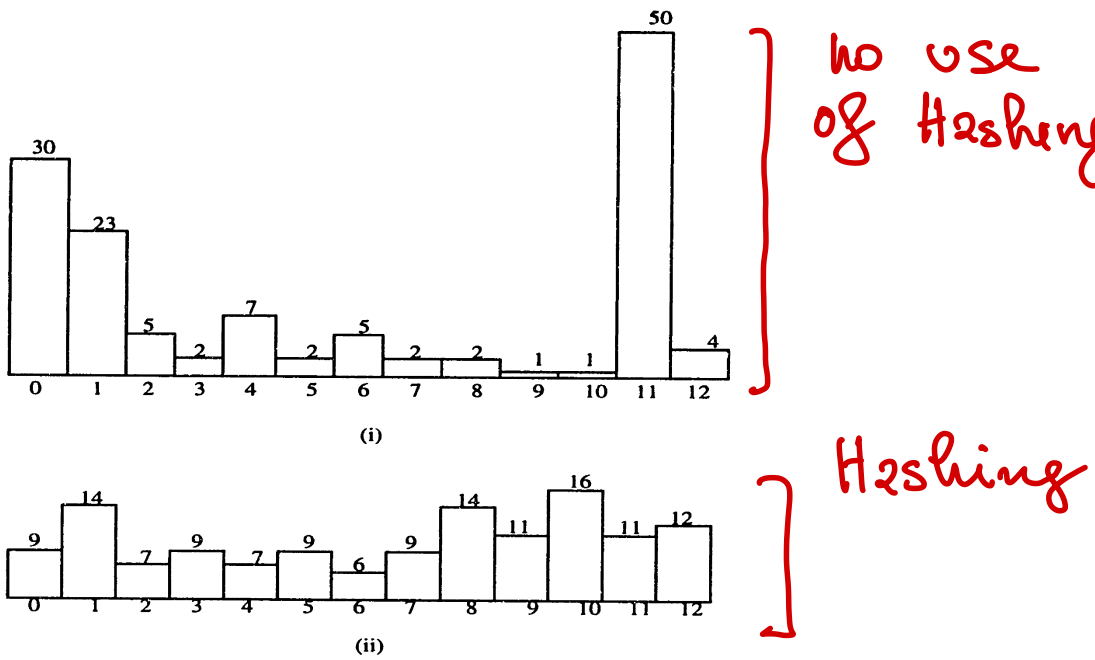
The Problem: Traditional Hashing

- In a distributed web cache, *Items* are mapped to n **(Cache) Servers** using:
- `server = hash(key) mod n`
- **The Fragility Issue:**
  - If a server fails $(n-1)$ or a new one is added $(n+1)$, nearly *all mappings change*.
  - **Result:** Massive *cache miss storm*, origin server overload, and "thrashing".

Traditional Hashing

(ii) $n = 13$ cache server $N = 134$ docs

$h(d) = ad + b \bmod 13$ (for fixed $a, b \in \mathbb{Z}_{13}$)



no use of Hashing

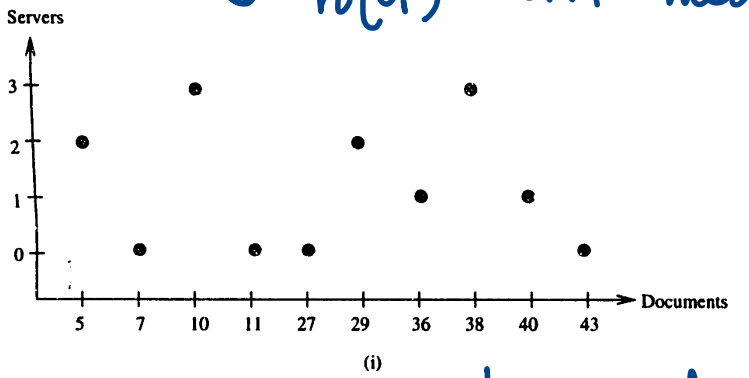
Hashing

Figure 2-2: This figure illustrates how hash functions distribute documents between servers. Assume that document names are integers, and that there are 13 servers $\{1, 2, \ldots, 13\}$. Documents are hashed to servers using a common type of hash function which is $f(d) = ad + b \pmod{13}$ for some fixed integers a and b. (i) The original distribution of 134 documents to servers. Note that some servers store many more documents than others, and thus in the model of equal access frequency they are more

Traditional Hashing: Reshuffling

RESHUFFLING when $n=4 \rightarrow n=5$

- $h(d) = d+1 \mod 4$



$\square \equiv h'(d) = d+1 \mod 5$

⇓

Almost every doc is moved to another SERVER

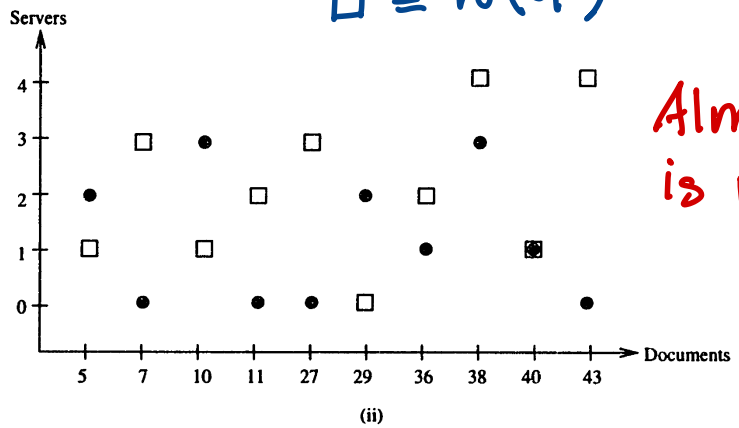


Figure 2-3: (i) The assignment of 10 documents to 4 servers using the hash function $f(d) = d + 1 \pmod 4$. (ii) The assignment after one additional server is added

RESHUFFLING AND SYNCHRONIZATION

Notification over Internet

When a Server is added to the System, **all** users should have to be **notified**, so that they can start to use the **same new** hash function.

Unrealistic Hypothesis

All Users should be notified of **every** updating of the Server Set **at the same time** : Such Global and Fast Synchronization is of course **NOT** possibile over the Internet.

Notification over Internet and Users' Views

Each user has its own **asynchronous View** of the available server set. Fig 2-3 can be also used to illustrate the problem of **asynchronous views**.

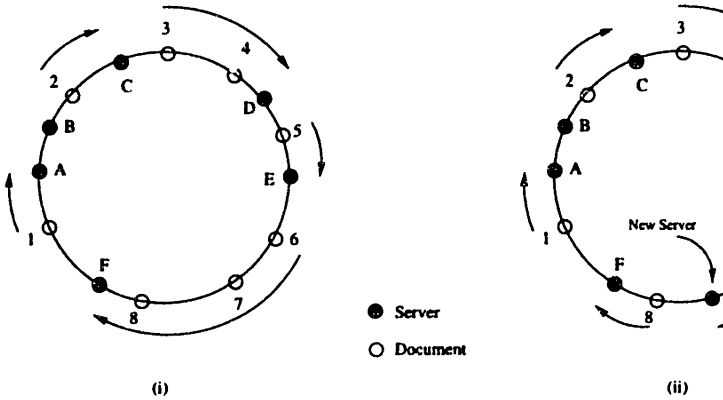
A Breakthrough: Consistent Hashing

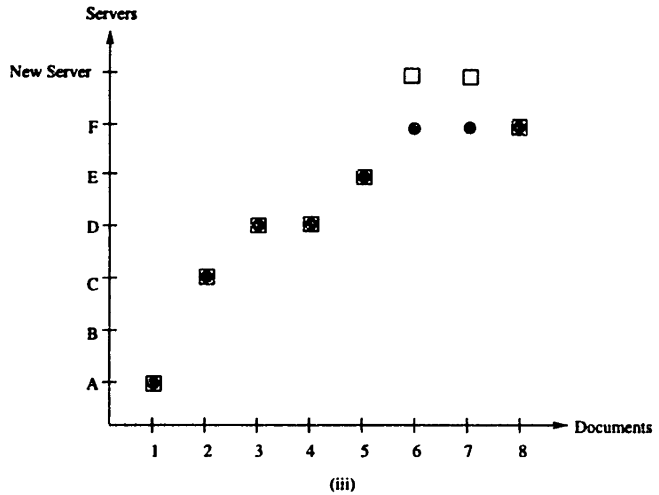
- Proposed by Karger, Lewin, et al. (1997) to solve the "view" inconsistency in Web Caching.
- **Core Concept:** Both **Cache** and **Items** are mapped onto the same mathematical space: the **Unit Circle** ($[0, 1)$).
- An Item is assigned to the first **Server** encountered moving clockwise on the circle.

Consistent Hashing

• SERVERS: A, B, C, D, E, F | ○ DOCS: 1, 2, ..., 8

2 → C; 3 → D → 4, ...





Efficiency: Smoothness and Balance

Consistent Hashing provides two critical properties:

1. Smoothness

When a Server is added or removed, only the Items that were/are mapped to that specific Server need to be moved. The rest of the mapping remains static.

2. Balance

Items are distributed among Servers such that no single Server is disproportionately overwhelmed, assuming a *uniform* hash function.

Handling Dynamic Views

- In a large-scale CLIENT-SERVER SYSTEMS, different Clients might see different sets of active Servers (different **Views**).
- **Spread:** Consistent Hashing ensures that a single Item is assigned to only a small number of distinct Servers across all possible Views.
- **Load:** Limits the total number of items any one Server is responsible for across different Views.

Solving the "Hotspot" Problem

- In traditional web traffic, certain content becomes viral or hot.
- Consistent Hashing prevents the **Hotspot** phenomenon.
- By decoupling the number of items from the number of (Physical) Servers, the system handles bursts of requests without localized failures.

PART III: Consistent Hashing

Formal Definitions: Items, Servers, Views, Hashing

- $C \equiv [0, 1)$ is the Unit Circle (the circle with circumference $= 1$)
- B is the set of *Servers* with $|B| = \ell$ and I is the set of *Items* with $|I| = n$
- A (client) *View* V is a subset of B; Let $\mathcal{V} = \{V_1, \ldots, V_k\}$ be a collection of Views.
- A *ranged-hash* function $f : 2^B \times I \rightarrow B$ maps any item i to a Server B in every possible view V: we use notation $f_V(i)$ instead of $f(V, i)$ and require that $f_V(I) \subseteq V$.
- A *ranged-hash family* $\mathcal{F}$ is a family of ranged-hash functions.

Properties of Consistent Hashing: Definitions

- **Monotonicity:** For each function $f \in \mathcal{F}$, and for all $V \subseteq W \subseteq \mathcal{V}$ we should have: if $\forall\, i \in I\ :\ f_W(i) \in V$, then $f_V(i) = f_W(i)$
- **Balance:** For any fixed view V, $i \in I$, $b \in V$ and $f \in_{\mathbf{u}} \mathcal{F}$, the items should be distributed "*randomly*", i.e., $\mathbb{P}_f(f_V(i) = b) = \tilde{O}(\frac{1}{|V|})$
- **Spread:** Let $\mathcal{V} = \{V_1, \ldots, V_k\}$. For a fixed i, the *Spread* of an item $i \in I$ should be "*small*", where $\text{Spread}(i) = |\{f_{V_j}(i)\}_{j=1\ldots k}|$
- **Load:** The (overall) *Load* of every $b \in B$ should be "*small*", where $\text{Load}(b) =$ the n. of distinct items i s.t. $f_V(i) = b$ for at least one view $V \in \mathcal{V}$

Properties of Consistent Hashing

- **Balance:** Standard properties of (Universal) Families of Hash Functions
- **Monotonicity:** Assume item set I is initially assigned to a Server view V and then some new Servers are added to form view W: to avoid large *reshuffling*, we require that items may move to *new* Servers **but not** to *old* different Servers.
  Remark: This property is not guaranteed by classic (universal) hash functions (i.e. $f_{a,b}(x) = ax + b \bmod p$)

Stability of Consistent Hashing

Lemma (Expected number of items that remain stable)

Let $\mathcal{F}$ be a monotonic, balanced ranged-hash family. Let V, W be two views. Then:

$$\mathbb{E}_{f \in_u \mathcal{F}}(|\{i \in I : f_V(i) = f_W(i)\}|/n) = \Omega\left(\frac{|V \cap W|}{|V \cup W|}\right)$$

Stability of Consistent Hashing

Proof's Key Ingredients.

- **First Fact:** Balancing implies that the *Expected Load* of every Server is almost the same. So when a Server is added (or removed) the same expected fraction of items can change location.
- Then, proceed in two steps.

i) Expand V into $V \cup W$: *monotonicity* says that only items assigned $W - V$ can move and the expected fraction of such items (thanks to *balancing*) is $O(|W - V|/|V \cup W|)$.

ii) Remove V from $V \cup W$: proceed similarly as (i) and get that expected fraction of such items is $O(|V - W|/|V \cup W|)$.

By combining (i) and (ii), the expected fraction of items that remain fixed is $1 - O(|W - V|/|V \cup W| + |V - W|/|V \cup W|) = \Omega\left(\frac{|V \cap W|}{|V \cup W|}\right)$ □

More about Spread

- **Spread** is the number of copies of each item i that the *Central Server* must assign to the set of Servers over all possible views! For each *responsible* Server of item i, the Central Server must supply a copy of i. So keeping Spread small is crucial to eliminate *Swamping*
- **Density-View Hypothesis (DVH): each view in $\mathcal{V}$ contains at least $|B|/t$ Servers. $t \geq 1$ is a parameter of the scheme.**
- **Spread** is very sensitive to the view family $\mathcal{V}$. Even assuming that each $V \in \mathcal{V}$ has size $|B|/t$, then a family of t views exists that forces the Spread of any item to be t: just take t disjoint views!
- **Key Question:** How does the Spread grow as a function of the number k of views?
  A: There are hash families whose spread grows linearly in k. We will build a *Consistent Hash Family* with Spread growing only **logarithmically** in k! :):)

More about Load

- **Load:** $\text{Load}_f(\mathcal{V}, b)$ of a Server b is the overall number of items that are assigned to b via $f_V(*)$ for all views $V \in \mathcal{V}$
- **Load** is related to *Spread*, but they are not equivalent: consider the extremal case where all items are mapped into the same Server in all views: Spread is zero, while Load is $|I|$. Clearly this is not a balanced hashing....
- **Load** is studied under the **DVH** with parameter t, as well: each view in $\mathcal{V}$ contains at least $|B|/t$ Servers

Consistent Hashing

Consistent-Hashing Families

A Family $\mathcal{F}$ of range-hash functions that have: Good Balance, Monotonicity, Low Spread and Low Load.

The analysis of a range-hash function family focuses on deriving explicit performance results for each of the *consistency* properties discussed before

Consistent Hashing: A Simple Efficient Solution (Informal)

The Uniform unit Circle Hash Family $\mathcal{UC}_{rnd}$

- **Mapping:** Pick two **random** functions $r_I : I \rightarrow C$ and $r_B : B \rightarrow C$ that map respectively I and B to *points* of the Unit Circumference C
- **Resource Assignment:** An item i is assigned to Server b (according to the hash function (r_I, r_B)) if $r_B(b)$ is the *first* Server point encountered when C is traversed *clockwise* from $r_I(i)$

Properties of the $\mathcal{UC}_{rnd}$ family (Informal)

- **Why Random Hashing?**
  Suppose an Adversary A has the power to crash f Servers/servers. If A knows the mapping of items and servers, then he can choose all servers in a portion of the circle: This yields a *gap* with no servers at all. Then all items will be re-assigned to the server located at the end of the *gap*: this server will get overloaded.
  If instead the mapping uses *randomness* and the Adversary A does not know the *outcome* of the random choices, then the above strategy does not work: he cannot create gaps with high probability.

Properties of the $\mathcal{UC}_{rnd}$ family (Informal)

- **Why Mapping over a Circle? Geometry helps!**

Properties of the $\mathcal{UC}_{rnd}$ family (Informal)

- **Why Mapping over a Circle? Geometry helps!**
- **Balance:** $\mathcal{UC}_{rnd}$ uses uniform distribution for both items and servers, the expected Load of every server will be well distributed.

Properties of the $\mathcal{U}C_{rnd}$ family (Informal)

- **Why Mapping over a Circle? Geometry helps!**
- **Balance:** $\mathcal{U}C_{rnd}$ uses uniform distribution for both items and servers, the expected Load of every server will be well distributed.
- **Monotonicity:** When a *new* server b is added, the *only* items re-assigned are those that are assigned to b

Properties of the $\mathcal{U}C_{rnd}$ family (Informal)

- **Why Mapping over a Circle? Geometry helps!**
- **Balance:** $\mathcal{U}C_{rnd}$ uses uniform distribution for both items and servers, the expected Load of every server will be well distributed.
- **Monotonicity:** When a *new* server b is added, the *only* items re-assigned are those that are assigned to b
- **Load and Views:** Since a server b will get only the items close to it, the load of b does not increase drastically with multiple views under the Density-View Hypothesis (**DWH**)

Properties of the $\mathcal{UC}_{rnd}$ family (Informal)

- **Why Mapping over a Circle? Geometry helps!**
- **Balance:** $\mathcal{UC}_{rnd}$ uses uniform distribution for both items and servers, the expected Load of every server will be well distributed.
- **Monotonicity:** When a *new* server b is added, the *only* items re-assigned are those that are assigned to b
- **Load and Views:** Since a server b will get only the items close to it, the load of b does not increase drastically with multiple views under the Density-View Hypothesis (**DWH**)
- **Spread and Views:** Since both items and servers are unif. distributed, only a small number of server points will be responsible for any fixed item: this implies that, even if there are many views, the *Spread* of any item will not increase dramatically with the number k of views.

Properties and Performances of the $\mathcal{UC}_{rnd}$ family: Summary

Properties and Performances of the $\mathcal{U}C_{rnd}$ family: Summary

- **Balance:** Items are distributed to servers uniformly at random.

Properties and Performances of the $\mathcal{U}C_{rnd}$ family: Summary

- **Balance:** Items are distributed to servers uniformly at random.
- **Monotonicity:** When a *new* server b is added, the *only* items re-assigned are those that are assigned to b

Properties and Performances of the $\mathcal{UC}_{rnd}$ family: Summary

- **Balance:** Items are distributed to servers uniformly at random.
- **Monotonicity:** When a *new* server b is added, the *only* items re-assigned are those that are assigned to b
- **Load and Views:** The Load of any server increases only **logarithmically** with the number k of views (under **DVH**) in contrast to **linearly** with *standard* hashing

Properties and Performances of the $\mathcal{UC}_{rnd}$ family: Summary

- **Balance:** Items are distributed to servers uniformly at random.
- **Monotonicity:** When a *new* server b is added, the *only* items re-assigned are those that are assigned to b
- **Load and Views:** The Load of any server increases only **logarithmically** with the number k of views (under **DVH**) in contrast to **linearly** with *standard* hashing
- **Spread and Views:** The *Spread* of any item will increase only **logarithmically** with the number k of views (under **DVH**) in contrast to **linearly** with *standard* hashing

PART II: RIGOROUS ANALYSIS OF THE $\mathcal{UC}_{rnd}$ FAMILY

Revising Definitions: Items, servers, Views, Hashing

- C is the Unit Circle (the circle with circumference $= 1$)
- B is the set of *servers* with $|B| = \ell$ and I is the set of *Items* with $|I| = n$
- A (client) *View* V is a subset of B; Let $\mathcal{V} = \{V_1, \ldots, V_k\}$ be a collection of Views.

Revising Definitions: Items, servers, Views, Hashing

Definition ($\mathcal{U}C_{rnd}$ Construction)

1). A *ranged-hash* function on C is a pair of functions (r_B, r_I):

$$r_B : B \times [\mathbf{m}] \to C \quad \text{and} \quad r_I : I \to C$$

Note 1: Each server b has m copies mapped to m points of C: m will be a parameter of the Data Structure.

2). The $\mathcal{U}C_{rnd}$ family is the set of all possible pairs (r_B, r_I) that maps servers and items to C.

KEY FACT: choosing a random pair from $\mathcal{U}C_{rnd}$ is equivalent to mapping (without replacement) both (the m copies of) every server and every item to random points of C, *independently and uniformly*.

Revising Definitions: Items, servers, Views, Hashing

Definition ($\mathcal{UC}_{rnd}$: Mapping Items to server Points in every View)

For each view $V \in \mathcal{V}$, consider the set of all points assigned to servers in V:

$$P_V = \bigcup_{b \in V} \{r_B(b,1), \ldots, r_B(b,m)\}$$

Each item $i \in I$ is assigned for the view V to the closest clockwise point assigned to a copy of some $b \in V$, i.e.,

$$f_V(i) = \text{server associated with } \text{argmin}_{p \in P_V}\{(p - r_I(i)) \pmod 1\}$$

Monotonicity

Theorem (I. Monotonicity)

$\mathcal{UC}_{rnd}$ is strictly monotone:
For each function $(r_B, r_I) \in \mathcal{UC}_{rnd}$, and for all $V \subseteq W \subseteq \mathcal{V}$, we have:
if $f_W(i) \in V$, then $f_V(i) = f_W(i)$

Proof.

Let $V \subseteq W$, hence $P_V \subseteq P_W$. Let $x = r_I(i)$. Let $p^* = \text{argmin}_{p \in P_W}(p - x) \pmod 1$.

1. If $f_W(i) \in V$, then point p^* that "captured" i in W actually belongs to a server in V.
2. Thus, $p^* \in P_V$.
3. Since P_V is a subset of P_W, and p^* was the *closest* successor in the larger set, it must remain the *closest* successor in the subset.
4. Therefore, $f_V(i) = f_W(i)$.

Useful Lemma for Spread and Load

- Consider the $\mathcal{UC}_{rnd}$ family with parameter $m \geq 1$, for a view collection $\mathcal{V} = \{V_1, \ldots V_k\}$ such that: (i) $|\cup_{j=1}^{k} V_j| = T$ and (ii) the *DVH* holds, i.e., $\forall j = 1, \ldots, k$, $|V_j| \geq T/t$ for some density parameter $t \geq 1$.
Let $N > 0$ our confidence parameter (typically N is a linear function of $n = |I|$ and/or $\ell = |B|$)

Lemma ((A) - Hitting Property)

Any fixed subset $S \subseteq C$ of measure $s \geq \frac{4t \log(2Nk)}{Tm}$ contains at least one server point from every view in $\mathcal{V}$, with probability at least $1 - 1/2N$ (the probability is over the rnd uniform choice of map r_B)

Typical Parameter Setting

- N is a linear/polynomial function of $n = |I|$ and/or $\ell = |B|$
- $T = \Theta(\ell)$ and $t = O(\mathrm{polylog}(n, \ell))$

Useful Lemma for Spread and Load

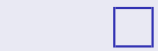
Proof of Lemma A.

- From the def. of $\mathcal{U}C_{rnd}$, we can assume that the points of the server are distributed *i.u.r.* over the circle C.
- For each fixed $j \in [k]$, let
$X_j = \textit{overall number of points in } S \textit{ associated to some server in } V_j$.
- Wlog, DVH implies there are $M = mT/t$ server points in each view V_j, so $\mathbb{E}(X_j) = M \cdot s$.
- Since X_j is a binomial distribution, we can apply Chernoff's Bound, and easily get that $\mathbb{P}(X_j = 0) \leq 2\exp(-Ms/4)$.

□

Proof of Lemma A - follows.

- $\mathbb{P}(X_j = 0) \leq 2\exp(-Ms/4) \quad [*]$.
- We want $2\exp(-Ms/4) = 1/(2Nk)$.
- So, setting $s = \frac{4t\log(2N\mathbf{k})}{Tm}$ in $[*]$, we have that, for any fixed $j \in [k]$, $\mathbb{P}(X_j = 0) \leq 1/(2Nk)$
- from Union Bound on k, the probability that subset $S \subseteq C$ has at least one server point *from every single view* V_j is at least $1 - 1/2N$.



Bounding the Spread of Items

Recall that the Spread of an item i over $\mathcal{V}$ is
$\text{Spread}_{f \in \mathcal{UC}_{rnd}}(\mathcal{V}, i) = |\cup_{j=1\ldots k} \{f_{V_j}(i)\}|$

Theorem ((B) - Bound on the Spread)

Under the **DVH** *with parameter* t*, any item* $i \in I$ *has* $\text{Spread}_{f \in \mathcal{UC}_{rnd}}(\mathcal{V}, i) = O(t \log(Nk))$*, with probability at least* $1 - 1/N$*, over the random choice* $f \in_u \mathcal{UC}_{rnd}$*.*

Key Fact

The Spread of any item increases only **logarithmically** in both the confidence parameter N and the number k of views in $\mathcal{V}$

Spread Analysis: Proof of Theorem B

Proof Strategy:

I) Define event $A \equiv$ *"The Theorem is false"* ;

II) Define a set $\mathcal{B}$ of few simple events and:
(1) Prove $A \implies \mathcal{B}$;
(2) Prove that *every* simple event in $\mathcal{B}$ has low probability.

Spread Analysis: Proof of Theorem B

Step II). Define a set of "few" events $\mathcal{B}$ s.t.: (1) $A \implies \mathcal{B}$; (2) *every* event in $\mathcal{B}$ has low probability.

- Fix an arc $S \subseteq C$ of length $s = 4t \log(4Nk)/Tm$ to the right of point $r_l(i)$. Our set $\mathcal{B}$ is formed by two events B_1 and B_2 where:
  - $\mathbf{B_1} \equiv$ "$\exists j \in [k]$ *s.t.* V_j has no server points in S"
  - $\mathbf{B_2} \equiv$ "$X > 8t \log(4Nk)$", where X is the r.v. counting the overall number of server points in S w.r.t. all views in $\mathcal{V}$.

Claim 1

$\neg B_1 \cap \neg B_2 \implies \neg A$

Spread Analysis: Proof of Claim 1

Claim 1

$\neg B_1 \cap \neg B_2 \implies \neg A$

Proof.

- If $\neg B_1$ occurs then *every view* V_j has at least one server point inside S: So, in every view V_j, item i will be assigned to some server having point *inside* S.
- The above fact easily implies that $\text{Spread}_{f \in \mathcal{UC}_{rnd}}(\mathcal{V}, i)$ *cannot be larger* than the overall number X of server points inside S.
- Then if $\neg B_2$ (i.e. $X \leq 8t \log(4Nk)$) occurs, we easily have $\neg A \equiv \text{Spread}_{f \in \mathcal{UC}_{rnd}}(\mathcal{V}, i) \leq 8t \log(4Nk)$

Spread Analysis: Back to Proof of Theorem B

We need to prove both B_1 and B_2 have negligible probability.
I). As for B_1, Lemma A (Hitting Property) easily implies that, under the **DVH**, $\mathbb{P}(B_1) \leq 1/2N$
II). As for B_2, since the overall number of server points, picked i.u.r over C, is $M = Tm$, we have $\mathbb{E}(X) = Tm \cdot (4t \log(4Nk)/Tm) = 4t \log(4Nk)$.
Again, X has binomial distribution: we can apply CB and get, for any $t \geq 1$: $\mathbb{P}(X > 2 \cdot 4t \log(4Nk)) \leq 2 \exp(-t \log(4Nk)) \leq 1/2N$
- Combining (I) and (II), we easily get:

$$\mathbb{P}(A) \leq \mathbb{P}(B_1 \cup B_2) \leq \mathbb{P}(B_1) + \mathbb{P}(B_2) \leq 1/2N + 1/2N = 1/N$$

Load Analysis

Recall the Key parameters

- $n = |I|$, n. of items ; $\ell = |B|$, n. of servers ; - $m =$ n. of copies of each server
- $k = |\mathcal{V}|$, n. of views
- $T = \cup_j |V_j|$, overall n. of servers used by all views in $\mathcal{V}$
- $t =$ the density parameter: each view $V \in \mathcal{V}$ has size $\geq T/t$

Load of a server

$\text{Load}_f(\mathcal{V}, b)$ of a server b is the overall number of items that are assigned to (any of the m copies of) b via $f_V(*)$ for all views $V \in \mathcal{V}$

Load Analysis

Recall the Key parameters

- $n = |I|$, n. of items ; $\ell = |B|$, n. of servers ; - $m =$ n. of copies of each server
- $k = |\mathcal{V}|$, n. of views
- $T = \cup_j |V_j|$, overall n. of servers used by all views in $\mathcal{V}$
- $t =$ the density parameter: each view $V \in \mathcal{V}$ has size $\geq T/t$

Load of a server

$\text{Load}_f(\mathcal{V}, b)$ of a server b is the overall number of items that are assigned to (any of the m copies of) b via $f_V(*)$ for all views $V \in \mathcal{V}$

Theorem ((C) - Bound on the Load)

Under the **DVH**, *for any server* $b \in B$, $Load_f(\mathcal{V}, b) = O(\frac{nt}{T} \log(Nmk))$, *with probability at least* $1 - 1/N$ *over the choice* $f \in_u \mathcal{U}C_{rnd}$.

Proof of Theorem C: An Overview

Key Ingredients

1. Thanks to the *Mapping Strategy*, an item assigned to a server (copy) b *must fall* into one of the m **arcs** the server's points are responsible for.
2. Lemma A (Hitting property) implies the expected length $\mathbb{E}(s)$ of the **arcs** above is quite *small*: $\frac{4t \log(2Nk)}{Tm}$
3. Since item points are mapped **i.u.r** on the circle, the expected number of items falling inside one of the m **arcs** of b can be bounded using the previous two steps

Proof of Theorem C: Formal Steps

- Fix one of the m points of b and call it p_b: consider the arc S_b starting from p_b and going counter-clockwise: we ensure that, *for any view in* $\mathcal{V}$, there is at least another bucket point in S with prob at least $1 - 1/(2Nm)$.
- Setting $N' = Nm$, Lemma A implies S will be not larger than $s = \frac{4t \log(2Nmk)}{Tm}$, with prob at least $1 - 1/2N'$).
- By union bound over all m points of b, we get that the event A = "the size of the arcs of *all* the m points of b is $s \leq \frac{4t \log(2Nmk)}{Tm}$ " has prob at least $1 - 1/2N$.
- We easily have that:
  $\mathbb{P}(\text{Load}_f(\mathcal{V}, b) \geq z) \leq \mathbb{P}(\text{Load}_f(\mathcal{V}, b) \geq z|A) \cdot 1 + 1 \cdot (2/N)$
- So, we need to bound $\mathbb{P}(\text{Load}_f(\mathcal{V}, b)|A)$: to do that, we next bound the expectation of the number X of items falling inside an arc of size s.
- We proceed as follows

Proof of Theorem C: Formal Steps

If Event A holds Then:

- Any item assigned to a server b must fall inside the arc of size s on the left of one of its m copies.
- in the worst-case pattern, all such m arcs are not overlapping. In this case, the overall region R covered by server b has size

$$m \cdot s = m \cdot \frac{4t \log(2Nmk)}{Tm} = \frac{4t \log(2Nmk)}{T}$$

- Let $X(b) =$ the number of items falling inside R
- Since each item is assigned i.a.u. over the circle, we easily have for

$$\mathbb{E}(X(b)) = n \cdot \frac{4t \log(2Nmk)}{T}$$

Proof of Theorem C: Formal Steps

Concentration on the expectation of $X(b)$

The proof now proceeds applying standard Chernoff's Bounds on the r.v. $X(b)$ since the points of the items are assigned i.u.r. over the circle (and are independent on the rnd points of the servers as well - Principle of Deferred Decisions)

Balance

Given any bucket $b \in B$, let $M(b)$ the overall measure of the set of points in C that b is responsible for.

Lemma

Let $V \in \mathcal{V}$. With Prob. $\geq 1 - 1/N$, all Servers $b \in B$ will have
$M(b) = O\left(\frac{1}{|V|} \cdot \left(1 + \frac{\log(N|V|)}{m}\right)\right)$

Proof.

1. Fix any Server point of $p_b \in C$ with $b \in V$ and look at the arc on the left determined by the next Server point you meet. Q. How large can be the union of all the m such arcs associated to b?
2. A. We can use the same arguments of previous lemmas and get that the measure of this union is $M(b) = O\left(\frac{1}{|V|} \cdot \left(1 + \frac{\log(N|V|)}{m}\right)\right)$ with Prob $\geq -1 - 1/(N|V|)$.
3. Take union bound over all buckets in V.

Balance

Lemma

Fix any $V \in \mathcal{V}$ and an Item $i \in I$. The the Prob. that i is mapped into one copy of b is $O\left(\frac{1}{|V|} \cdot \left(1 + \frac{\log(N|V|)}{m}\right)\right) + \frac{1}{N}$

Proof.

Immediate consequence of previous Lemma. □

Lemma

Under the **DVH**, *the Prob. that any item i is mapped into (one copy of) of any fixed server b, in at least one view $V \in \mathcal{V}$ is,*
$O\left(\frac{t \log(Nmk}{T}\right) + \frac{1}{N}$